NEXT ENGINEER · _SHEERAT_

Object Oriented System Design with C++

BCS054

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 5 | **Credits:** 3

L-T-P: 3-0-0 | **Scheme:** AKTU B.Tech 3rd year, Departmental Elective, 2024-25 onwards

UNIT 1 Object Orientation and UML

UNIT 2 Structural and Behavioural Modelling

UNIT 3 Object Oriented Analysis and Design

UNIT 4 C++ Basics and Functions

UNIT 5 Classes, Objects, Inheritance and Polymorphism

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: comparison tables, UML notation and design steps.
3. **Solved examples**: diagrams described in words, and C++ programs with their output.
4. **Code / notation to remember**: what you must be able to draw or write.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Units 1 and 2. OO concepts, modelling principles, UML building blocks and the nine classic diagrams. Practise one class diagram, one use case diagram and one sequence diagram for an ATM or library system.
- **Afternoon (2 hours):** Unit 3. Rumbaugh's OMT (object, dynamic, functional models), OO analysis and design steps, SA/SD and JSD, mapping OO designs to C.
- **Evening (3 hours):** Units 4 and 5. C++ program structure, references, inline, default arguments, overloading, friend and virtual functions; constructors and destructors, operator overloading, inheritance types and run-time polymorphism. Then the "Last-hour checklist".
- **In the exam:** draw UML diagrams neatly with correct symbols and labels. For C++ questions, write a complete small program and show its output.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Object Orientation and UML

✍️ 2-mark definitions

- **Object orientation:** organizing software as a collection of discrete objects that combine data structure and behaviour.
- **Object:** a concept, abstraction or thing with identity, state (attribute values) and behaviour (operations).
- **Class:** a description of a group of objects with the same attributes, operations, relationships and semantics.
- **Abstraction:** focusing on the essential aspects of an entity and ignoring unimportant details.
- **Encapsulation (information hiding):** separating the external interface of an object from its internal implementation.
- **Inheritance:** sharing attributes and operations among classes based on a hierarchical (generalization) relationship.
- **Polymorphism:** the same operation may behave differently on different classes.
- **Identity:** each object is distinct, even if its attribute values are the same as another's.
- **Model:** an abstraction of something for understanding it before building it.
- **UML (Unified Modeling Language):** a standard visual language for specifying, visualizing, constructing and documenting software systems, created by Booch, Rumbaugh and Jacobson and standardized by the OMG.
- **Things, relationships and diagrams:** the three building blocks of UML.
- **Stereotype:** a UML extension mechanism that creates a new kind of element from an existing one, written «name».

💡 Core concepts

Characteristics of OO approach: identity, classification, inheritance and polymorphism (Rumbaugh); plus abstraction, encapsulation, combining data and behaviour, sharing (reuse), and emphasis on what an object is rather than how it is used.

Principles of modelling (Booch et al.)

1. The choice of models deeply affects how a problem is attacked and how a solution is shaped.
2. Every model can be expressed at different levels of precision.
3. The best models are connected to reality.
4. No single model is enough; a system is best approached through a small set of nearly independent models.

UML building blocks

Block	Kinds
Structural things	class, interface, collaboration, use case, active class, component, node

Block	Kinds
Behavioural things	interaction (messages), state machine
Grouping things	package
Annotational things	note
Relationships	dependency, association, generalization, realization
Diagrams	class, object, component, deployment (structural); use case, sequence, collaboration, state chart, activity (behavioural)

UML relationship notation

Relationship	Notation	Meaning
Association	solid line	structural link between objects
Aggregation	line with hollow diamond at the whole	"has-a", parts can exist alone
Composition	line with filled diamond at the whole	strong "part-of", parts die with the whole
Generalization	line with hollow triangle at the parent	"is-a" inheritance
Realization	dashed line with hollow triangle	a class implements an interface
Dependency	dashed arrow	one element uses another

Architecture of UML (4 + 1 views): use case view (centre), design view, process view, implementation view and deployment view.

Mechanisms in UML: specifications, adornments, common divisions (class vs object, interface vs implementation) and extensibility mechanisms (stereotypes, tagged values, constraints).

🧮 Solved examples

Example 1 (class notation). A class box has three compartments: name (Account), attributes (- accNo: int, - balance: float) and operations (+ deposit(amt: float), + withdraw(amt: float): bool). Visibility: + public, - private, # protected, ~ package.

Example 2 (aggregation vs composition). A Department has Professors (aggregation, hollow diamond: professors exist without the department). A House has Rooms (composition, filled diamond: rooms are destroyed with the house).

Key points to remember

- Multiplicity notation: 1, 0..1, *, 1..*, 2..5.
- Generalization arrow points to the parent.
- UML is a language, not a method; it does not tell you the process.

⭐ Important questions

1. Explain object orientation and its characteristics (identity, classification, inheritance, polymorphism).
2. What is modelling? State the principles of modelling and the importance of modelling.
3. Explain the conceptual model of UML: building blocks, rules and common mechanisms.
4. Explain the relationships in UML with notation and examples.
5. Explain the architecture (4 + 1 views) of UML.

NEXT ENGINEER
sheerat

UNIT 2 Structural and Behavioural Modelling

✍️ 2-mark definitions

- **Class diagram:** shows classes, interfaces and their relationships; the static design view.
- **Object diagram:** a snapshot of objects and links at a particular moment.
- **Use case:** a description of a sequence of actions a system performs that gives a measurable result to an actor.
- **Actor:** a role played by a user or external system that interacts with the system.
- **«include» and «extend»:** include means the base use case always uses another use case; extend means the extending use case adds optional behaviour at an extension point.
- **Sequence diagram:** an interaction diagram that shows messages between objects ordered in time, with lifelines and activation bars.
- **Collaboration (communication) diagram:** an interaction diagram that shows objects and links, with numbered messages, emphasizing structure.
- **State chart (state machine) diagram:** shows the states of an object, the events that cause transitions, and actions.
- **Activity diagram:** shows the flow of control from activity to activity, including decisions, forks and joins, and swimlanes.
- **Component diagram:** shows the physical software components (executables, libraries, files) and their dependencies.
- **Deployment diagram:** shows the hardware nodes and the components deployed on them.
- **Swimlane:** a partition of an activity diagram showing which actor or object is responsible for each activity.

💡 Core concepts

Sequence vs collaboration diagram

Point	Sequence	Collaboration
Emphasis	time order of messages	structural organization of objects
Order shown by	vertical position	sequence numbers 1, 1.1, 2 ...
Lifeline / activation	yes	no
Links shown	no	yes
Conversion	semantically equivalent; one can be converted to the other	

Activity vs state chart diagram

Point	Activity	State chart
Focus	flow of activities (process)	life cycle of one object
Nodes	activities and actions	states
Transitions	on completion of activity	triggered by events
Concurrency	fork and join bars	concurrent sub-states

State chart elements: initial state (filled circle), final state (bull's-eye), states (rounded rectangles) with entry/exit/do actions, transitions labelled event [guard] / action, composite and history states.

Activity diagram elements: start and end nodes, action states, decision diamonds with guards, merge, fork and join bars for parallel flow, swimlanes, object flow.

🧮 Solved examples

Example 1 (use case diagram for an ATM). Actors: Customer, Bank (system). Use cases: Withdraw Cash, Deposit Cash, Check Balance, Transfer Funds, all «include» Authenticate (enter card and PIN); Print Receipt «extend»s Withdraw Cash. The system boundary is a rectangle labelled ATM.

Example 2 (sequence diagram for withdrawal, in words). Lifelines: Customer, ATM, Bank. Messages: 1 insertCard() → ATM; 2 enterPIN() → ATM; 3 verify(card, PIN) → Bank; 4 ok ← Bank; 5 withdraw(amount) → ATM; 6 debit(account, amount) → Bank; 7 success ← Bank; 8 dispenseCash() → Customer; 9 ejectCard().

Example 3 (state chart for an order). States: Placed → (pay) → Paid → (ship) → Shipped → (deliver) → Delivered (final). From Placed or Paid, the event cancel goes to Cancelled (final).

Example 4 (activity diagram for online exam registration). Start → Login → decision [valid?] → (no) Show error → back to Login; (yes) Fill form → fork: Upload photo and Pay fee in parallel → join → Generate admit card → End. Swimlanes: Student, System, Payment gateway.

Notation to remember

- Lifeline: object box with a dashed vertical line; activation: thin rectangle on it.
- Synchronous message: solid arrow with a filled head; return: dashed arrow; asynchronous: open arrowhead.
- Use case: ellipse; actor: stick figure; system boundary: rectangle.
- Component: rectangle with the component icon; node: 3-D box.

⭐ Important questions

1. Draw a class diagram (and an object diagram) for a library / hospital / banking system.
2. Draw a use case diagram with include and extend relationships for a given system.
3. Draw sequence and collaboration diagrams for a scenario and compare them.
4. Draw state chart and activity diagrams for a given process and explain their elements.
5. Explain component and deployment diagrams with examples.

UNIT 3 Object Oriented Analysis and Design

✍️ 2-mark definitions

- **Object-oriented analysis (OOA):** understanding the problem by finding the objects, classes and relationships in the application domain (what the system must do).
- **Object-oriented design (OOD):** deciding how to build the system: system architecture, subsystems and detailed class design.
- **OMT (Object Modelling Technique):** Rumbaugh's methodology using three models: object, dynamic and functional.
- **Object model:** the static structure: classes, attributes, operations and relationships (class diagrams).
- **Dynamic model:** the behaviour over time: events and states (state diagrams, event traces).
- **Functional model:** the data transformations: how values are computed (data flow diagrams).
- **System design:** high-level decisions: subsystems, concurrency, allocation to hardware, data storage, control strategy and trade-offs.
- **Object design:** detailed design of classes, algorithms, associations and data structures.
- **SA/SD (Structured Analysis / Structured Design):** a function-oriented approach using data flow diagrams, data dictionary and structure charts.
- **JSD (Jackson Structured Development):** a method that models the real world as entities with sequential processes, then builds a network of communicating processes.
- **Implementation in non-OO language:** translating classes into structures and functions, e.g. in C.

💡 Core concepts

OMT phases

1. **Analysis:** build the object, dynamic and functional models from the problem statement.
2. **System design:** split into subsystems, identify concurrency, allocate to processors, manage data stores, handle global resources, choose control (procedure-driven, event-driven, concurrent), handle boundary conditions.
3. **Object design:** refine classes, design algorithms, optimize access paths, implement control and associations, adjust inheritance.
4. **Implementation:** translate to code.

Steps in building the object model: identify classes (nouns in the problem statement), prepare a data dictionary, identify associations (verbs), identify attributes, organize using inheritance, verify access paths, iterate, and group into modules.

OO vs structured (SA/SD) approach

Point	SA/SD	OO
Focus	functions and data flow	objects (data and behaviour together)

Point	SA/SD	OO
Decomposition	top-down by function	by objects and classes
Main diagram	DFD, structure chart	class, sequence, state diagrams
Reuse	low	high (inheritance, classes)
Change handling	changes in function ripple through	localized in classes

JSD vs OMT: JSD models the world as time-ordered entity actions and is suited to concurrent, event-oriented systems; OMT gives a broader static, dynamic and functional view.

Mapping OO concepts to a non-OO language (C)

- Class → struct for data plus functions that take a pointer to the struct.
- Encapsulation → separate .h (interface) and .c (implementation) files; static functions for private parts.
- Inheritance → embed the parent struct as the first member of the child struct.
- Polymorphism (dynamic binding) → function pointers in the struct (a manual "virtual table").
- Associations → pointers or arrays of pointers.

Object-oriented programming style: reusability (keep methods small and coherent, keep methods consistent, separate policy and implementation), extensibility (encapsulate classes, hide data structures, avoid case statements on object type), robustness (protect against errors, validate arguments), and programming-in-the-large (avoid premature coding, document, keep methods understandable).

🧮 Solved examples

Example 1 (finding classes for an ATM problem). Problem: "Customers use bank cards at ATMs to withdraw cash from their accounts. The bank keeps accounts and records transactions." Nouns: Customer, Card, ATM, Cash, Account, Bank, Transaction. Keep Customer, Card, ATM, Account, Bank, Transaction; discard Cash (it is an attribute value, not a class). Associations: Customer owns Card; Card accesses Account; Bank holds Account; ATM communicates with Bank; Transaction records changes to Account.

Example 2 (class in C).

```
typedef struct Shape Shape;
struct Shape { double (*area)(Shape *self); };        /* "virtual" function */

typedef struct { Shape base; double r; } Circle;      /* inherits Shape */
double circleArea(Shape *s) { Circle *c = (Circle *)s; return 3.14159 * c->r * c->r; }

/* Circle c = { { circleArea }, 2.0 };  c.base.area(&c.base) returns 12.566 */
```

Key points to remember

- Object model answers "what", dynamic model "when", functional model "how values are computed".
- In OMT, the object model is the most important and is built first.
- SA/SD diagrams: DFD (levels 0, 1, 2), data dictionary, structure chart.

⭐ Important questions

1. Explain OMT and its three models with examples.
2. Explain the steps of object-oriented analysis and system design.
3. Explain object design: refining classes, designing algorithms and associations.
4. Compare SA/SD and JSD with the OO approach.
5. Explain how OO concepts are mapped to a non-OO language like C; explain OOP style guidelines.

NEXT ENGINEER
sheerat

UNIT 4 C++ Basics and Functions

✍️ 2-mark definitions

- **C++:** a general-purpose language developed by Bjarne Stroustrup (1979, Bell Labs) that adds classes and other OO features to C.
- **Namespace:** a named scope that prevents name clashes, e.g. std; used with `using namespace std;` or `std::cout`.
- **cin and cout:** standard input and output stream objects, used with >> (extraction) and << (insertion).
- **Reference variable:** an alias for an existing variable, declared with &, e.g. `int &r = x;`. It must be initialized and cannot be re-seated.
- **Call by reference:** passing a reference so the function can modify the caller's variable.
- **Inline function:** a function whose body the compiler may insert at the call site to avoid call overhead; suitable for small functions.
- **Default arguments:** values used for parameters when the caller omits them; they must be the rightmost parameters.
- **Function overloading:** several functions with the same name but different parameter lists (number or types).
- **Friend function:** a non-member function given access to the private and protected members of a class, declared with friend inside the class.
- **Virtual function:** a member function declared virtual in the base class so that calls through a base pointer or reference use the derived class's version at run time.
- **Scope resolution operator (::):** accesses a global variable hidden by a local one, or defines a member function outside its class.
- **const qualifier:** makes a variable read-only; a const member function cannot change the object.

💡 Core concepts

Structure of a C++ program: documentation comments, preprocessor directives (#include), namespace declaration, class declarations, member function definitions, and the main() function.

C vs C++

Point	C	C++
Paradigm	procedural	multi-paradigm (procedural and OO)
Data security	no access control	private, protected, public
Overloading	not supported	function and operator overloading
I/O	printf, scanf	cin, cout
Memory	malloc, free	new, delete
References	no	yes

Point	C	C++
Namespaces, exceptions, templates	no	yes

Control flow: if, if-else, switch, for, while, do-while, break, continue, goto, return.

Pointer vs reference

Point	Pointer	Reference
Can be NULL	yes	no
Re-assignable	yes	no (always the same object)
Initialization	optional	required
Syntax for use	*p, p→	like the variable itself

Inline restrictions: the compiler may ignore inline for functions with loops, recursion, static variables, switch or goto.

Overloading resolution: exact match → promotion (char to int, float to double) → standard conversion (int to double) → user-defined conversion. Ambiguity is a compile error. Return type alone cannot distinguish overloads.

🧮 Solved examples

Example 1 (swap by reference).

```
#include <iostream>
using namespace std;
void swapRef(int &a, int &b) { int t = a; a = b; b = t; }
int main() {
    int x = 5, y = 9;
    swapRef(x, y);
    cout << x << " " << y;   // 9 5
}
```

Example 2 (default arguments and overloading).

```
float area(float r) { return 3.14f * r * r; }
int area(int l, int b = 2) { return l * b; }
// area(3.0f) → 28.26   area(4) → 8   area(4, 5) → 20
```

Example 3 (scope resolution).

```
int x = 10;
int main() { int x = 20; cout << x << " " << ::x; }   // 20 10
```

Example 4 (friend function).

```
class Box { int w; public: Box(int a) : w(a) {} friend int width(Box b); };
int width(Box b) { return b.w; }      // allowed: friend
// cout << width(Box(7));   prints 7
```

⌨️ Code to remember

```
// Inline function, namespace and virtual function in one program
#include <iostream>
using namespace std;

namespace geo { inline double sq(double x) { return x * x; } }

class Shape {
public:
    virtual double area() { return 0; }
    virtual ~Shape() {}
};
class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() override { return 3.14159 * geo::sq(r); }
};

int main() {
    Shape *s = new Circle(2);
    cout << s->area();     // 12.5664  (derived version chosen at run time)
    delete s;
}
```

⭐ Important questions

1. Explain the structure of a C++ program; compare C and C++.
2. Explain namespaces, references and call by reference with examples.
3. Explain inline functions, default arguments and function overloading.
4. What is a friend function? Write a program using a friend function of two classes.
5. Explain virtual functions and how they support run-time polymorphism.

UNIT 5 Classes, Objects, Inheritance and Polymorphism

✍️ 2-mark definitions

- **Class and object in C++:** a class defines data members and member functions; an object is a variable of that class type.
- **Access specifiers:** private (only inside the class and friends), protected (also in derived classes), public (everywhere). Default for class is private; for struct, public.
- **Constructor:** a member function with the class name and no return type, called automatically when an object is created. Types: default, parameterized, copy, and constructors with default arguments.
- **Copy constructor:** creates an object as a copy of another: ClassName(const ClassName &obj).
- **Destructor:** a member function ~ClassName() called automatically when an object is destroyed; it has no arguments and cannot be overloaded.
- **this pointer:** a pointer to the object on which a member function was called.
- **Static data member:** shared by all objects of the class; defined once outside the class.
- **Operator overloading:** giving additional meaning to an operator for user-defined types, using `returnType operator symbol (args)`.
- **Inheritance:** creating a derived class from a base class to reuse and extend it.
- **Virtual base class:** a base class declared virtual in the inheritance list so that only one copy is inherited in diamond (hybrid) inheritance.
- **Pure virtual function and abstract class:** `virtual void f() = 0;` makes the class abstract; it cannot be instantiated.
- **Compile-time vs run-time polymorphism:** function and operator overloading are resolved at compile time; virtual functions are resolved at run time (dynamic binding).

💡 Core concepts

Types of inheritance: single (A → B), multiple (A, B → C), multilevel (A → B → C), hierarchical (A → B, A → C) and hybrid (combination, e.g. diamond).

Visibility of inherited members

Base member	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

Order of constructors and destructors: base class constructors run first (in the order listed in the inheritance list; virtual bases first), then member objects, then the derived constructor. Destructors run in the exact reverse order.

Operators that cannot be overloaded: `.` (member access), `.*`, `::`, `?:` and `sizeof`. Overloading cannot change precedence, associativity or the number of operands.

Overloading as member vs friend: a binary operator as a member function takes one argument (the left operand is *this); as a friend it takes two. Friends are needed when the left operand is not a class object (e.g. `2 + c` or `cout << c`).

Early vs late binding

Point	Early (static) binding	Late (dynamic) binding
Decided at	compile time	run time
Mechanism	overloading, normal functions	virtual functions, vtable
Speed	faster	slightly slower
Flexibility	less	more

🧮 Solved examples

Example 1 (constructor and destructor order).

```
class A { public: A() { cout << "A "; } ~A() { cout << "~A "; } };
class B : public A { public: B() { cout << "B "; } ~B() { cout << "~B "; } };
int main() { B b; }
// Output: A B ~B ~A
```

Example 2 (copy constructor).

```
class Point {
    int x, y;
public:
    Point(int a, int b) : x(a), y(b) {}
    Point(const Point &p) : x(p.x), y(p.y) { cout << "copy "; }
    void show() { cout << x << "," << y; }
};
// Point p1(2, 3); Point p2 = p1; p2.show();   prints: copy 2,3
```

Example 3 (operator overloading for complex numbers).

```
class Complex {
    double re, im;
public:
    Complex(double r = 0, double i = 0) : re(r), im(i) {}
    Complex operator+(const Complex &c) const { return Complex(re + c.re, im + c.im); }
    friend ostream& operator<<(ostream &os, const Complex &c) {
        return os << c.re << " + " << c.im << "i";
    }
};
// Complex a(2, 3), b(1, 4); cout << a + b;   prints 3 + 7i
```

Example 4 (diamond problem with a virtual base class).

```
class Person  { public: int id = 1; };
class Student : virtual public Person { };
class Employee: virtual public Person { };
class TA : public Student, public Employee { };
// TA t; cout << t.id;   works: only one Person sub-object exists.
// Without "virtual", t.id is ambiguous (two copies).
```

Example 5 (static member).

```
class Counter { public: static int count; Counter() { count++; } };
int Counter::count = 0;
// Counter a, b, c; cout << Counter::count;   prints 3
```

⌨️ Code to remember

```
// Abstract class and run-time polymorphism through base pointers
#include <iostream>
using namespace std;

class Employee {
protected:
    string name;
public:
    Employee(string n) : name(n) {}
    virtual double salary() = 0;                  // pure virtual
    void show() { cout << name << ": " << salary() << endl; }
    virtual ~Employee() {}
};
class Manager : public Employee {
public:
    Manager(string n) : Employee(n) {}
    double salary() override { return 90000; }
};
class Intern : public Employee {
public:
    Intern(string n) : Employee(n) {}
    double salary() override { return 15000; }
};

int main() {
    Employee *staff[] = { new Manager("Asha"), new Intern("Ravi") };
    for (Employee *e : staff) { e->show(); delete e; }
}
// Asha: 90000
// Ravi: 15000
```

⭐ Important questions

1. Explain classes and objects in C++; explain access specifiers and the this pointer.
2. Explain constructors (default, parameterized, copy) and destructors with order of execution.

3. What is operator overloading? Overload binary + and << for a Complex class; list operators that cannot be overloaded.
4. Explain types of inheritance with examples; explain virtual base classes and the diamond problem.
5. Explain polymorphism: compile-time vs run-time; pure virtual functions and abstract classes.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ OO characteristics; four principles of modelling.
- ☐ UML building blocks; 4 relationships and their arrows; aggregation vs composition.
- ☐ 4 + 1 views; stereotypes, tagged values, constraints.
- ☐ Class, object, use case (include vs extend), sequence vs collaboration.
- ☐ State chart vs activity; fork/join; swimlanes; component and deployment.
- ☐ OMT: object, dynamic, functional models; analysis → system design → object design.
- ☐ SA/SD and JSD vs OO; mapping classes to C with structs and function pointers.
- ☐ C vs C++; reference vs pointer; inline; default arguments; overloading rules.
- ☐ Friend function; virtual function; scope resolution.
- ☐ Constructor types; copy constructor; destructor order (A B ~B ~A).
- ☐ Operator overloading (+ and <<); operators that cannot be overloaded.
- ☐ Inheritance types and visibility table; virtual base class; pure virtual and abstract class.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Object-Oriented Modeling and Design	James Rumbaugh et al.	Pearson Education	WorldCat
2	The Unified Modeling Language User Guide	Grady Booch, James Rumbaugh, Ivar Jacobson	Pearson Education	WorldCat
3	Object Oriented Programming with C++	E. Balagurusamy	McGraw-Hill Education	WorldCat
4	C++ Programming Black Book	Steven Holzner	Dreamtech Press	WorldCat
5	Object Oriented Programming in Turbo C++	Robert Lafore	Galgotia Publications	WorldCat
6	Object Oriented Programming with ANSI and Turbo C++	Ashok N. Kamthane	Pearson Education	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
7	C++: The Complete Reference	Herbert Schildt	McGraw-Hill Education	WorldCat

Syllabus source: AKTU B.Tech 3rd year syllabus, 2024-25 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER
sheerat